CS3281 / CS5281

Filesystems

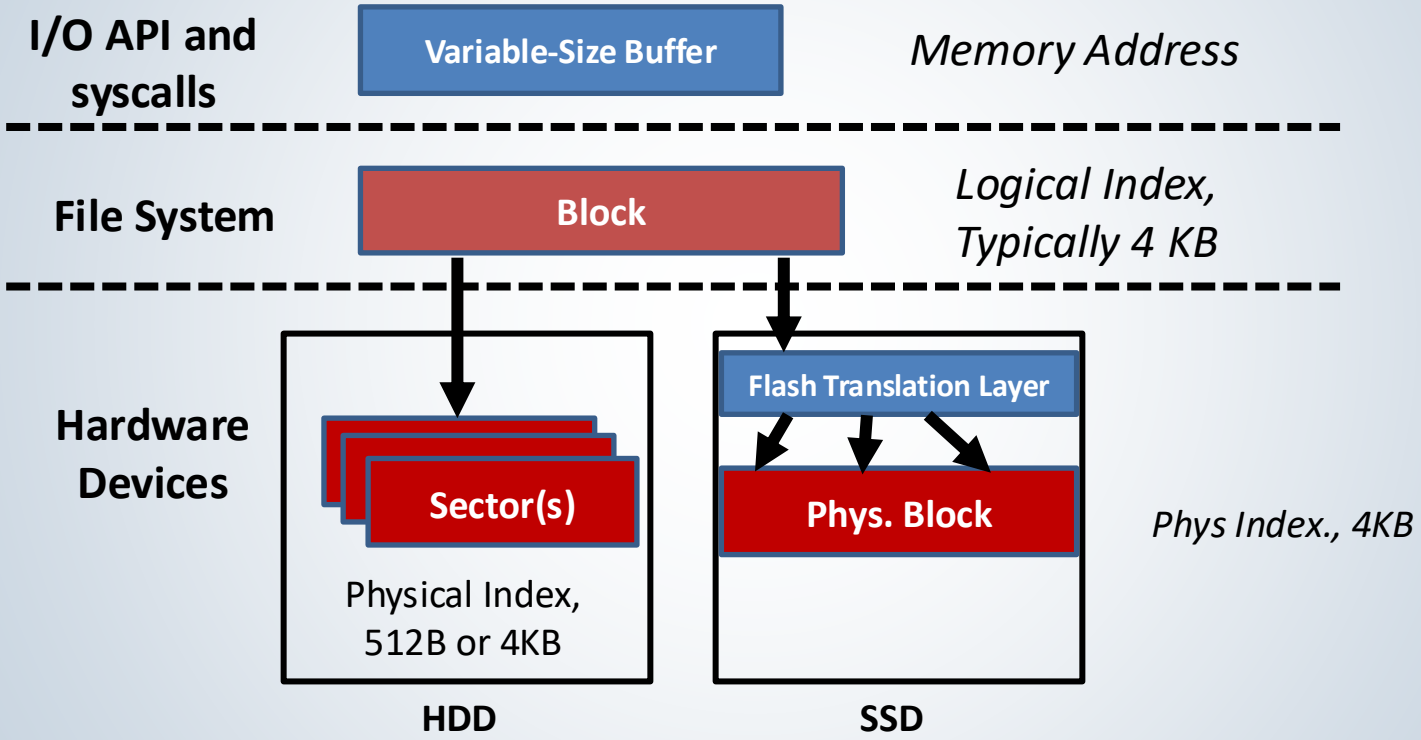
CS3281 / CS5281

Spring 2026

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

VANDERBILT UNIVERSITY

Introducing file systems



Classic OS situation

- Take limited hardware interface (array of blocks) and provide a more convenient/useful interface with:
  1. Naming: Find file by name, not block numbers
  2. Organization: Organize file names with directories
  3. Translation: Map files to blocks
  4. Protection: Enforce access restrictions
  5. Reliability: Keep files intact despite crashes, hardware failures, etc.

- We combine all of this to create a filesystem
  - Many different approaches and tradeoffs
  - FAT32, NTFS, ext4, ZFS, etc.

Overview

- A filesystem is an organized collection of files and directories
- The Linux kernel maintains a single hierarchical directory structure to organize all files in the system
  - Not like Windows where each drive (C, D, E, etc) has its own hierarchy
- Root directory is named /
  - Pronounced “slash”

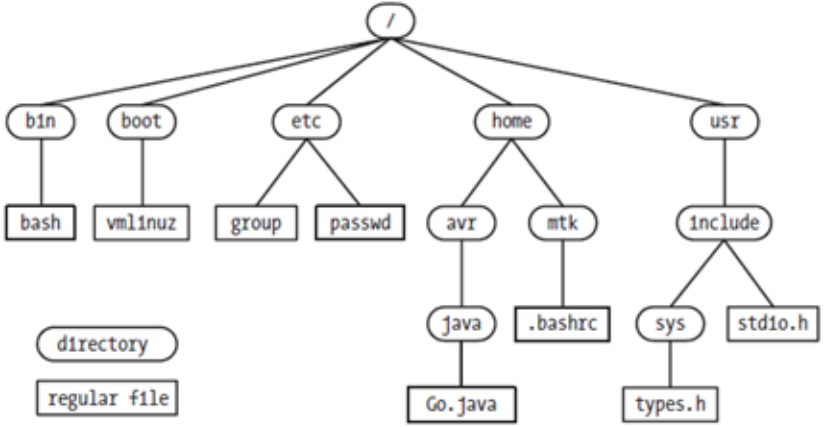


Figure 2-1: Subset of the Linux single directory hierarchy

*Figure from *The Linux Programming Interface* by Michael Kerrisk

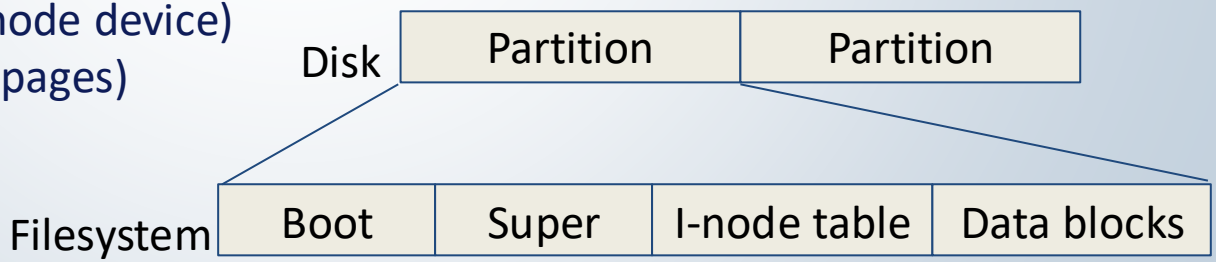
File Types

- Every file has a type
  - This is the character in the first column when you do ls -l
- Regular files: ordinary data files, like text files, executables, libraries
- Special files: files other than ordinary data files
  - Devices: represents a device (virtual or physical)
    - Block device (e.g., disk)
    - Character device (keyboard)
  - Directories
  - Symbolic links
- Example on right: block device files

```
daniel@ubuntu:/dev$ ls -l sda*
brw-rw---- 1 root disk 8, 0 Nov  5 09:21 sda
brw-rw---- 1 root disk 8, 1 Nov  5 09:21 sda1
daniel@ubuntu:/dev$
```

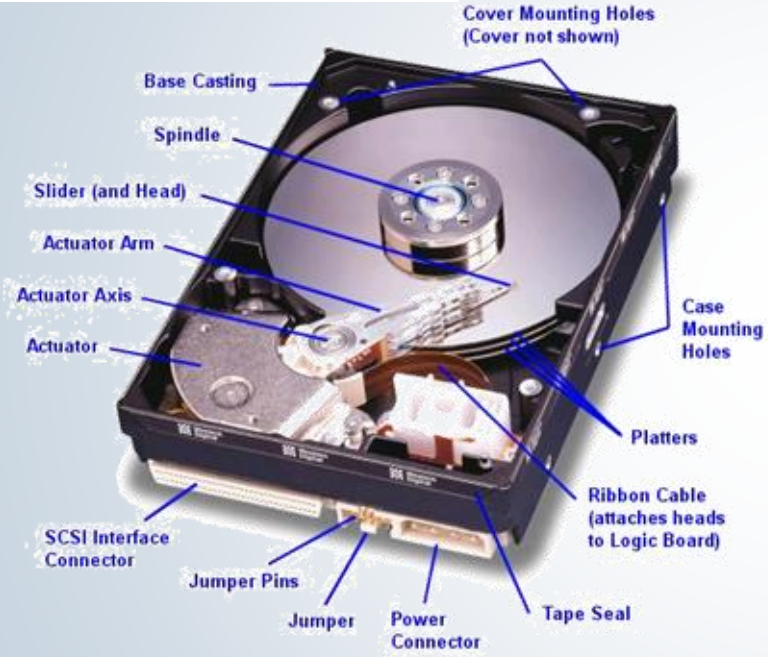
Back to Filesystems

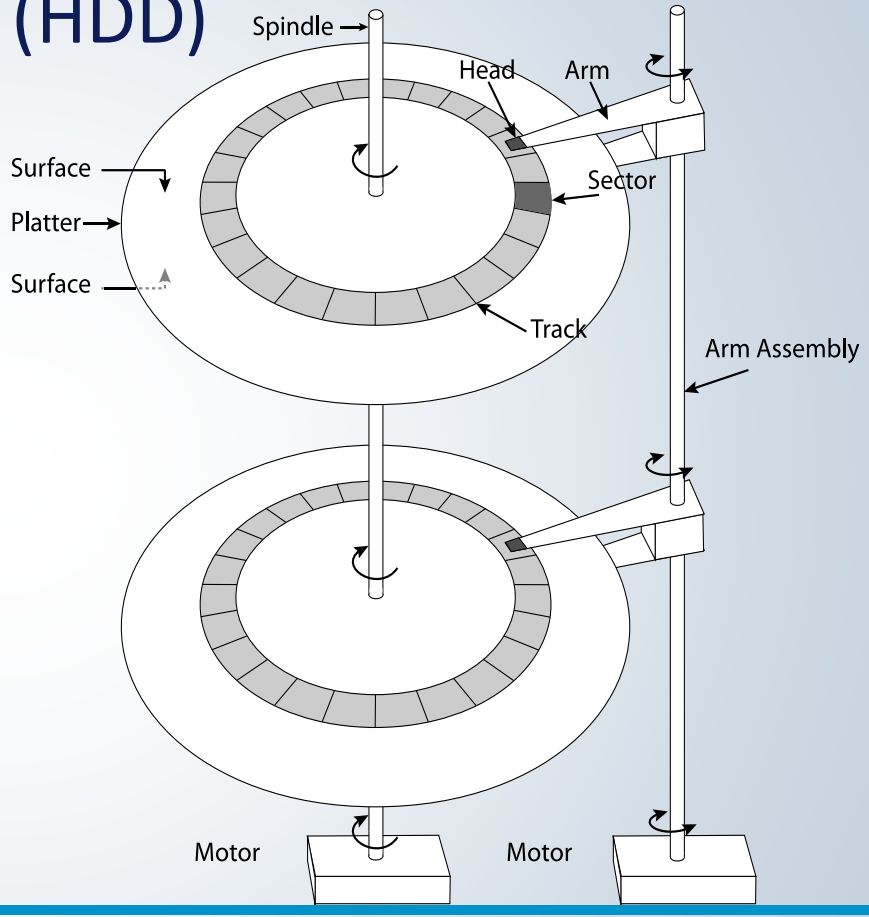
- A disk drive is divided into circles called tracks
  - Tracks are divided into sectors
    - Sectors are a series of physical blocks
      - Physical block: the smallest unit a disk can read or write
        - Usually 512 bytes (older disks) or 4096 bytes (newer disks)
- Each disk is divided into partitions
  - Each is a separate device under /dev
  - A partition holds either
    - *Filesystem* (on-disk structures)
    - Data area (raw-mode device)
    - Swap (for virtual pages)



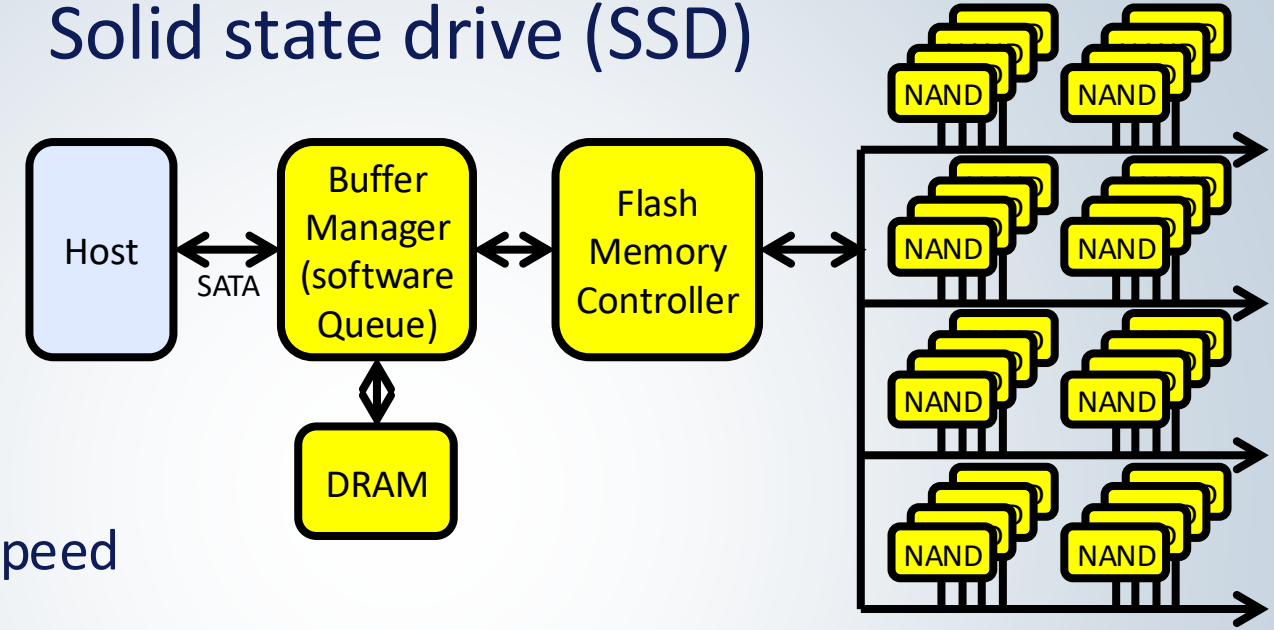
Hard Disk Drive (HDD)

Western Digital Drive
http://www.storagereview.com/guide/

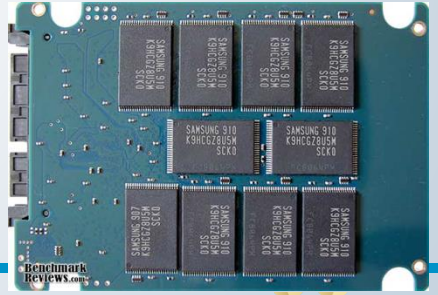




Solid state drive (SSD)



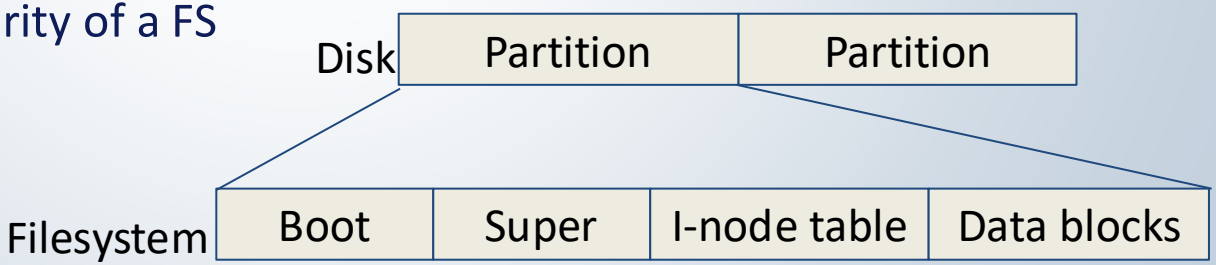
- Flash memory
- No issues with random access speed
- Writes are a concern though!
  - Writes 10x slower than reads
  - Limited write lifetime (~1-10k writes per page)



Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

Filesystem Structure

- Boot block: always the first block in a filesystem (FS)
  - Not used by FS; contains info to boot the OS
  - Only one needed by OS
- Super block: contains parameter info about the filesystem
  - Size of the i-node table, size of logical blocks, size of the filesystem (in logical blocks)
- I-node table: contains one (unique) entry for every file in file system
  - Contains most of the “metadata” about individual file
- Data blocks: the (logical) blocks that contain the data for files and directories
  - This is the vast majority of a FS



I-Nodes

- Index nodes (i-nodes) contains the following metadata about a file
  - File type (for example, regular, char device, block device, directory, symbolic link)
  - Owner of the file
  - Group of the file
  - File access permissions for three categories: user (owner), group, other
  - Three timestamps:
    - Time of last access (ls -lu)
    - Time of last modification (default timestamp in ls -l)
    - Time of last status change (change to i-node info) (ls -lc)
  - Number of hard links (pathnames) to file
  - Size of the file (in bytes)
  - Number of blocks allocated to file
  - Pointers to the data blocks

Directories

- A directory is stored in a filesystem in a similar way as a regular file, but
  - It is marked as a directory in its i-node
  - It's a file with a special organization: it's a table consisting of filenames and i-node numbers
- Example is on the right
- Note: the i-node doesn't have a filename!
  - Implication: you can have multiple links to the same file!

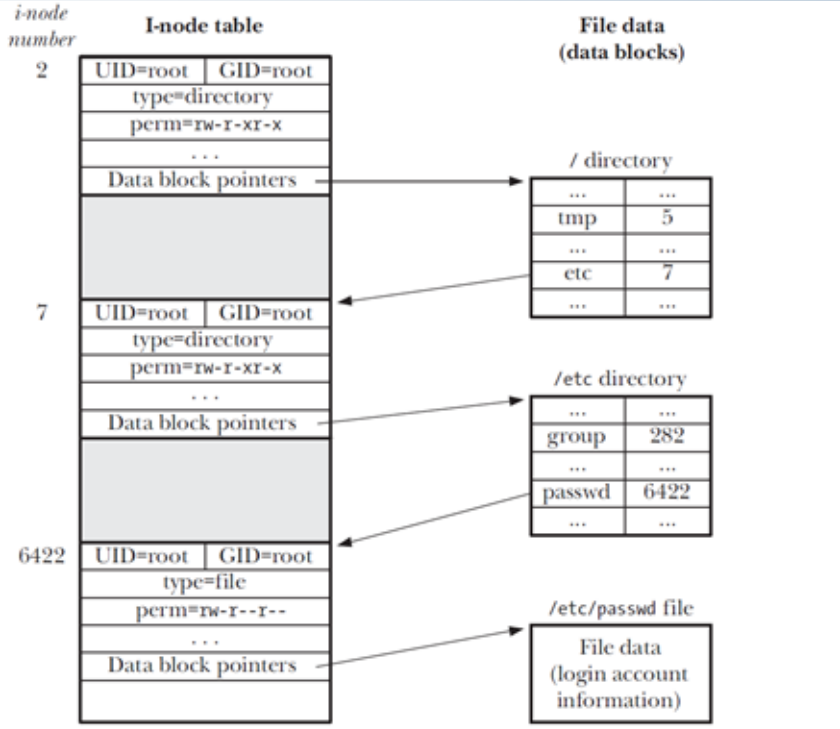


Figure 18-1: Relationship between i-node and directory structures for the file /etc/passwd

*Figure from *The Linux Programming Interface* by Michael Kerrisk

Data blocks

- How can files of very different sizes be supported?
  - One method: store pointers to the data blocks!
- Figure on the right shows how ext2 does this
  - Small files might fit entirely in direct pointers
- Bigger files use:
  - Indirect pointers
  - Double-indirect pointers
  - Triple-indirect pointers
- Advantages of pointers
  - Fixed-size i-node
    - But arbitrary size files
  - Store blocks non-contiguously

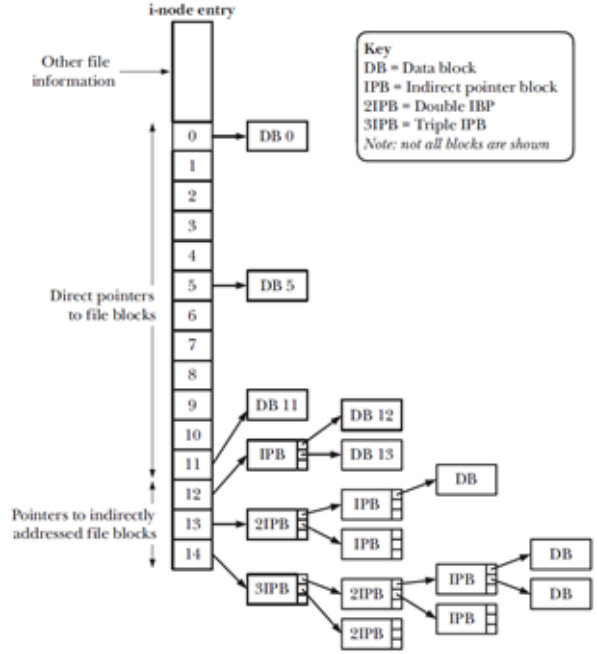
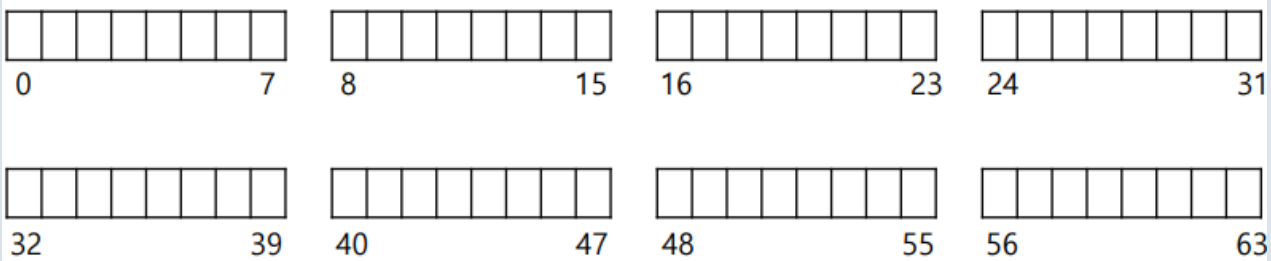


Figure 14-2: Structure of file blocks for a file in an *ext2* file system

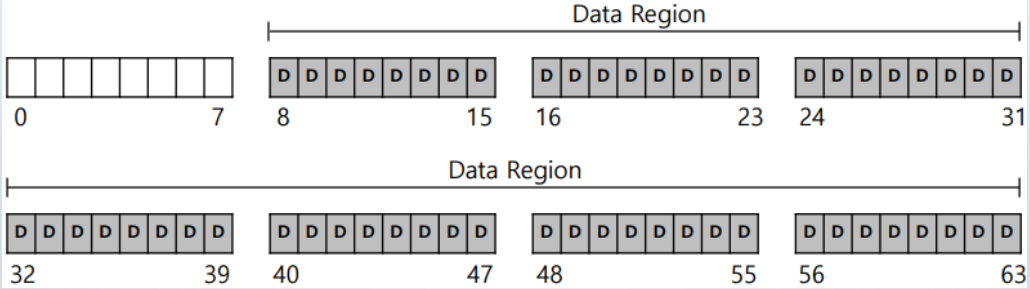
*Figure from *The Linux Programming Interface* by Michael Kerrisk

Very Simple File System (VSFS) Data Structures

- Divide disk into blocks
- Use one block size (4KB)
- Blocks are addressed from 0 to *N*-1 (N is the number of blocks)

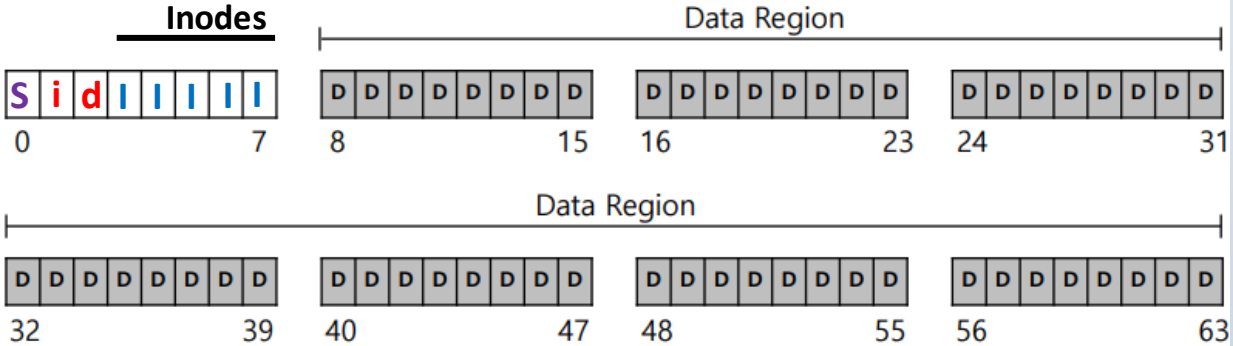


- Store user data in *data region* (e.g., files and directories)



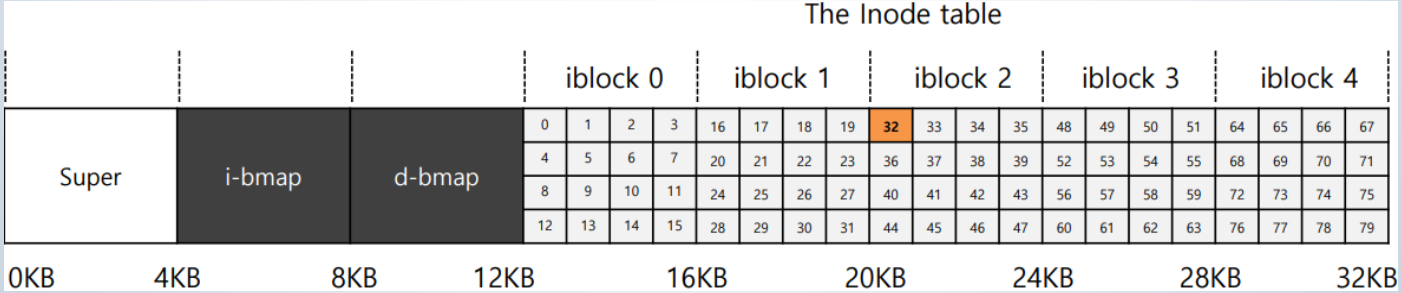
VSFS Data Structures (cont.)

- File system tracks information (metadata) about files.
  - E.g., a subset of data blocks that form a file, file size, access rights
- Metadata is stored in a structure called *inode*
  - inode table is an array of inodes
  - 256-byte inode
  - 16 inodes per block; 80 inodes in 5 blocks
- Use bitmap for tracking free inodes and data blocks
  - A bit indicates whether an inode or a block is free.
- Superblock stores information about a certain file system.
  - E.g., number of inodes and data blocks, the start of inode table



The I-Node

- Index node (inode): array of nodes is indexed
- Each inode is identified by an i-number
  - Used for indexing an array of inodes
- Find the byte address for the inode with i-number 32
  - Compute the offset into the inode table: 32 * sizeof(inode) = 8192 (8KB)
  - Add the offset to start address of inode table: 12KB + 8KB = 20KB
- inodes are fetched using *sectors* (a block consists of sectors)
  - 512-byte sectors
  - Sector number: (20 * 1024)/512 = 40



Open and read example

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[1]
open("/foo/bar")			read							
						read				
				read						
							read			
					read					
read()					read					
					write					
read()					read					
									read	
					write					
read()					read					
										read
					write					

time

find file inode

Open and read example

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[1]
open("/foo/bar")			read							
						read				
				read						
							read			
					read					
read()					read					
								read		
					write					
read()					read					
									read	
					write					
read()					read					
										read
					write					

time

update attributes

Open and read example

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[1]
open("/foo/bar")			read							
						read				
				read						
							read			
					read					
read()					read					
					write					
read()					read					
									read	
					write					
read()					read					
										read
					write					

time

read next file block

Open and read example

time

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[1]
open("/foo/bar")			read							
						read				
				read						
							read			
					read					
read()					read					
					write					
read()					read					
									read	
					write					
read()					read					
										read
					write					

inode reads/writes occur in memory

Create and write a file

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[1]
			read							
						read				
				read						
							read			
create (/foo/bar)		read								
		write	2							
							write	3		
					read					
					write	4				
				write						
write()					read					
	read									
	write									
								write		
					write					
write()					read					
	read									
	write									
									write	
					write					
write()					read					
	read									
	write									
										write
					write					

Create:

1. First, read the parent directory to ensure that name is not already used.
2. Find & claim a free inode.
3. Add <“name”, inode#> to parent directory.
4. Fill-in file metadata.

Create and write a file

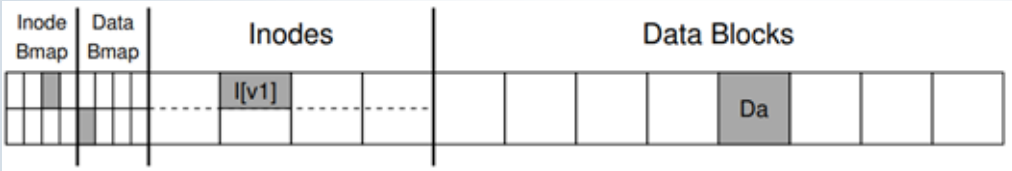
	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[1]
create (/foo/bar)		read write	read 2	read	read write	read	read	1	write 3	
				write	4					
write()	read write				read write			write		
write()	read write 2				read write	1 4			write 3	
write()	read write				read write					write

Write:

1. Look for remaining space in existing blocks first.
2. Find & claim a new data block.
3. Write data to new block
4. Point to it in inode

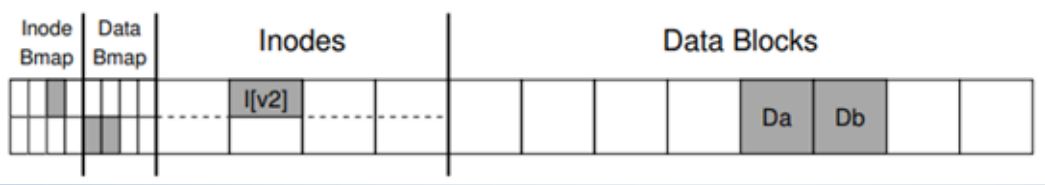
Crash Scenario

- Consider the following simple filesystem scenario from the textbook
  - One file with one allocated data block




```
owner        : remzi
permissions  : read-write
size         : 1
pointer      : 4
pointer      : null
pointer      : null
pointer      : null
```

- Suppose we want to append to this file; we have to
  - (1) write the data to a new data block, (2) update the i-node, (3) update the data bitmap
    - This requires three (separate) writes to disk
  - For example, we want the updated filesystem to look like this:




```
owner        : remzi
permissions  : read-write
size         : 2
pointer      : 4
pointer        5
pointer      : null
pointer      : null
```

Crash Scenario (cont.)

- But what if the system crashes after only 1 of the writes is performed?
  - **Just the data block is written**: the data is there, but the i-node doesn't know about it
  - **Just the i-node is updated**: the data isn't there, so we'll read garbage from disk
    - And the filesystem is inconsistent: the i-node (danglingly) points to a data block, but the data block bitmap thinks it hasn't been allocated.
  - **Just the data block bitmap is updated**: no i-node points to the data block, so there's a space leak

- What if the system crashes after 2 writes (out of 3) are performed?
  - **I-node and bitmap updated, data block not updated**: file contains garbage
  - **I-node and data block are written**: the bitmap thinks the block hasn't been allocated
  - **Bitmap and data block are written**: no i-node points to the data (so the data is effectively lost!). This is a data leak.

i: inode

dm: data (bit) map

db: data block

Action Sequence	Failure After First Action	Failure After Two Actions
1: i->db->dm	Inode points to the garbage data because data block is not updated. Inconsistent file system	Inode is updated but data map retains its old value. Because data map says that space is not allocated, the user data is lost. Inconsistent file system
2: i->dm->db	See Case 1, Failure After First Action Inconsistent file system	See Case 1, Failure After First Action Consistent file system
3: db->i->dm	Data block has content but neither the inode nor the data map has any information about it. This is called **data leak**. Consistent file system	See Case 1, Failure After Two Actions Inconsistent file system
4: db->dm->i	See Case 3, Failure After First Action Consistent file system	See Case 3, Failure After First Action Inconsistent file system
5: dm->i->db	Data map says that space is allocated but there is nothing in the block nor does any inode point to it. This is called **space leak**. The space does not have any data or has garbage data. Inconsistent file system	See Case 1, Failure After First Action and Case 5, Failure After First Action Consistent file system
6: dm->db->i	See Case 5, Failure After First Action Inconsistent file system	See Case 3, Failure After First Action Inconsistent file system

The Crash Consistency Problem

- Crashes can leave the filesystem in a bad post-crash state. This may be either:
  - Metadata inconsistency: filesystem structures disagree
  - Data loss despite consistency: filesystem metadata is self-consistent, but the data is lost

- Disk performs one write at a time.
  - Crashes or power loss occur between writes.

A Solution: Filesystem Checker

- A filesystem checker (fsck) scans the filesystem for inconsistencies and repairs them
  - Runs before filesystem is made available to users. It
    - Check the superblock; if it looks corrupted, you can try to use an alternate copy
    - Scan the i-nodes to see which blocks are in use and resolve inconsistencies in the allocation bitmaps
    - Check that each i-node is ok
    - Scan the entire directory tree (starting at root dir) and verify link counts
    - Check for bad blocks (e.g., data pointer points to an invalid address); just clear the bad pointers
    - Check that directories are consistent (e.g., contain "." and "..", there are i-nodes for each file entry)

Journaling

- A filesystem check can be very slow, especially for large filesystems
- An alternative is journaling, also called write-ahead logging
- Simple idea: before updating the on-disk structures, write information about the update to a *journal* (also called a log)
  - If there's a crash before the disk can be updated, the journal contains enough information to recover
    - You don't have to scan the entire disk!

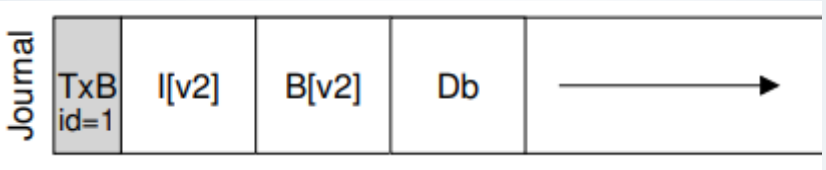
Example: ext2 vs. ext3

- The basic layout of ext2 (no journaling):

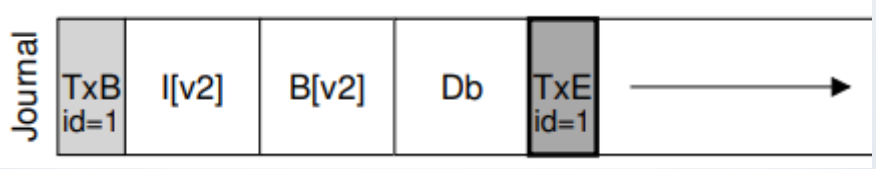
- The basic layout of ext3 (with journaling):

Data Journaling Phases

- Four basic phases:
  - Journal write: write contents of transaction to journal; wait for these to complete



  - Journal commit: write the transaction commit block; wait for write to complete; transaction is committed



  - Checkpoint: write the contents of the update (metadata and data) to disk

Timeline for Data Journaling

Journal contents				File System	
TxB	(metadata)	(data)	TxE	Metadata	Data
issue complete	issue complete	issue complete			
			issue complete		
				issue complete	issue complete

any order

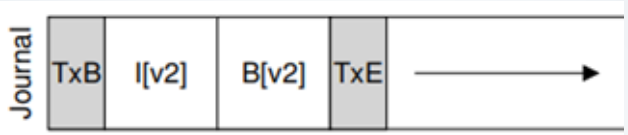
Recovery

- When a crash happens before *journal commit* completes, pending updates in a transaction are skipped.

- When a crash happens after *journal commit* completes and before *checkpoint* completes, transactions are replayed (redo logging)
  - File system finds out transactions that were committed successfully
  - The blocks of those specific transactions are written to their final disk locations again

Ordered (Metadata) Journaling

- Instead of journaling both data and metadata, just do metadata



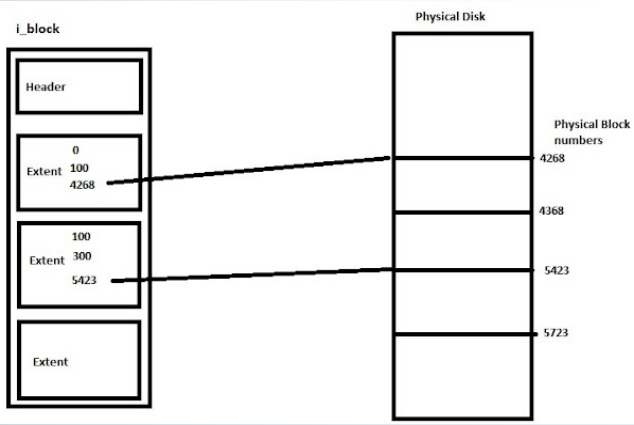
- In this case, our journaling protocol is:
  - **Data write**: write data to its final location
    - If we crash after this step but before the rest, only the data is lost. Transaction is discarded.
  - **Journal metadata** write: write metadata to the journal
  - **Journal commit**: write the transaction commit block
  - **Checkpoint metadata**: write the metadata update to final location in filesystem

Timeline for Metadata Journaling

TxB	**Journal contents** (metadata)	TxE	**File System** Metadata	Data
issue	issue			Issue complete
complete				
	complete			
		issue complete		
			issue complete	

Extents

- Recall that ext2 and ext3 use indirect pointers to data blocks (Figure on right)
  - Can be inefficient!
- The ext4 filesystem uses extents
  - An extent specifies an (1) an initial block address, and (2) the number of blocks in the extent
    - Large file will have multiple extents



http://ext2read.blogspot.com/2010/03/how-ext4-extents-work-earlier-ext2-and.html

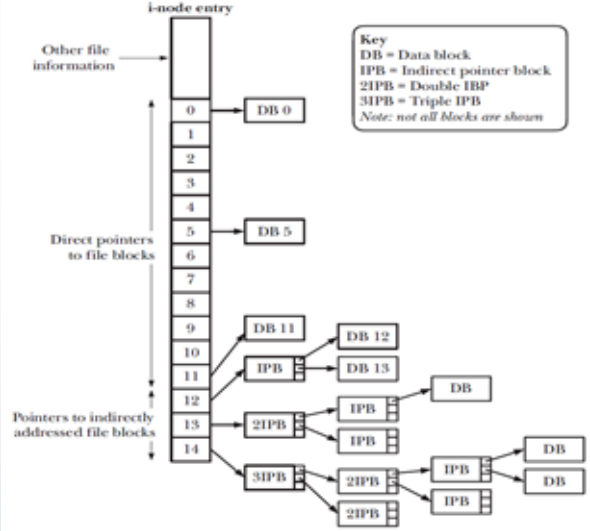


Figure 14-2: Structure of file blocks for a file in an *ext2* file system

*Figure from *The Linux Programming Interface* by Michael Kerrisk